



Unidad 04 – Condicionales

Cátedra: Programación en Computación – UTN FRRQ Docentes: Longhi Pablo, Talijancic Iván



De qué se trata esta unidad

Hasta ahora tus programas hacían **siempre lo mismo**: leían datos, calculaban y mostraban. La clase pasada aprendiste a preguntar (`measured >= lowerLimit && measured <= upperLimit`), pero la respuesta sólo se imprimía.

Hoy esa respuesta **decide**.

Al terminar vas a poder:

-  Ejecutar código **sólo si** se cumple una condición, con `if`
-  Elegir entre dos caminos con `if / else`
-  Encadenar varios casos con `else if`, **en el orden correcto**
-  Usar `switch-case` cuando comparás contra valores fijos
-  Saber **cuál de los dos** conviene en cada situación



Nada de hoy es nuevo del todo. Las condiciones son las mismas de la unidad 03. Lo único que cambia es que ahora, en vez de imprimirse, gobiernan qué líneas se ejecutan.

1. `if`: ejecutar sólo si

La forma mínima. Si la condición es `true`, se ejecuta el bloque; si es `false`, se saltea.

```
const temperature = 91

if (temperature > 85) {
  console.log('⚠ Temperatura fuera de rango')
}

console.log('Fin del control')
```

```
⚠ Temperatura fuera de rango
Fin del control
```

Si la temperatura fuera `70`, la primera línea **no se imprimiría** y el programa saltaría directo a `Fin del control`.

1.1 Cómo se lee

```
if (condición) {  
  // ↑           ↑  
  // |           | el bloque: se ejecuta sólo si la condición es true  
  // └─ una expresión que da true o false  
}
```

Adentro del paréntesis va **cualquier cosa que dé true o false**: una comparación, una combinación con `&&` o `||`, o una variable que ya guarde un booleano.

```
const isWithinRange = measured >= lowerLimit && measured <= upperLimit  
  
if (isWithinRange) {  
  console.log('Medición correcta')  
}
```

💡 **Guardar la condición en una variable con nombre descriptivo** hace que el `if` se lea como una oración. Es la diferencia entre `if (isWithinRange)` y un paréntesis de tres renglones.

1.2 ⚠ Las llaves van siempre

JavaScript te deja omitirlas cuando el bloque tiene una sola línea:

```
if (temperature > 85) console.log('Alarma')
```

En la cátedra no se acepta. El motivo es concreto: el día que agregues una segunda línea, va a parecer que está adentro del `if` y no lo va a estar.

```
// ❌ El programa dice "Alarma" siempre, aunque la temperatura esté bien  
if (temperature > 85) console.log('Alarma')  
  console.log('Se registró el evento')
```

La segunda línea está **fuera** del `if`, por más que la indentación diga lo contrario. Con llaves, el error no puede ocurrir.

2. ↔ if / else : uno u otro

Cuando hay exactamente dos caminos y **siempre** se toma uno de los dos.

```
const measured = 405
const upperLimit = 399

if (measured > upperLimit) {
  console.log('Tensión ALTA')
} else {
  console.log('Tensión dentro del límite superior')
}
```

Tensión ALTA

El `else` **no lleva condición**: es "en cualquier otro caso".

🔑 Con `if` / `else` es **imposible** que no se imprima nada. Uno de los dos bloques se ejecuta sí o sí.

3. 🏗️ `if` / `else if` / `else` : varios casos

Para más de dos posibilidades. Se evalúan **de arriba hacia abajo** y **se ejecuta el primero que dé `true`**; el resto se saltea.

```
const measured = 372.5

if (measured < 361) {
  console.log('BAJA')
} else if (measured <= 399) {
  console.log('NORMAL')
} else {
  console.log('ALTA')
}
```

NORMAL

3.1 🔑 Sólo se ejecuta UNA rama

Es la propiedad más importante de la estructura. Apenas una condición da `true`, se ejecuta su bloque y **se sale**: las de abajo ni se evalúan.

Por eso la segunda condición es `measured <= 399` y no `measured >= 361 && measured <= 399`. Si el programa llegó hasta ahí, **ya sabemos** que no es menor a 361 — la primera condición dio `false`. Escribir de nuevo esa mitad es redundante.

3.2 ⚠️ El orden importa muchísimo

Este es **el** error de la unidad. Mismos rangos, orden invertido:

```
const measured = 300

if (measured <= 399) {
  console.log('NORMAL')
} else if (measured < 361) {
  console.log('BAJA')
} else {
  console.log('ALTA')
}
```

NORMAL

300 V no es normal, es baja. Pero como `300 <= 399` da `true`, entra por la primera rama y la segunda **nunca se evalúa**.

Peor todavía: el programa **no falla**. No hay error, no hay aviso. Simplemente clasifica mal, siempre, y te enterás cuando alguien mira el informe.

🔑 **La regla: de lo más específico a lo más general.** O, con rangos numéricos, **ordenados de menor a mayor** (o de mayor a menor), sin saltar. Si escribís los rangos en orden, el problema no puede aparecer.

3.3 El `else final` es opcional... y conviene ponerlo

```
if (temperature > 85) {
  console.log('ALARMA')
} else if (temperature > 70) {
  console.log('ATENCIÓN')
}
```

Si la temperatura es `50`, este programa **no imprime nada**. A veces es lo que querés; casi siempre es un olvido.

💡 Poné el `else final` aunque sólo diga "todo normal". Te obliga a pensar qué pasa con los casos que no contemplaste, y evita programas mudos.

4. 🧑‍🔧 Condiciones anidadas

Un `if` puede vivir adentro de otro:

```
const isEnergized = true
const temperature = 91

if (isEnergized) {
  if (temperature > 85) {
    console.log('Tablero energizado con sobret temperatura')
  }
}
```

Funciona, pero **casi siempre hay una forma más simple**:

```
if (isEnergized && temperature > 85) {
  console.log('Tablero energizado con sobret temperatura')
}
```

🔑 Si un **if** anidado no tiene **else**, se puede reemplazar por **&&**. Menos llaves, menos indentación y se lee de corrido. Anidá sólo cuando cada nivel tiene su propio **else**.

5. switch-case

Cuando comparás **una misma variable** contra **varios valores exactos**, la cadena de **else if** se vuelve repetitiva:

```
if (maintenanceCode === 'P') {
  console.log('Preventivo')
} else if (maintenanceCode === 'C') {
  console.log('Correctivo')
} else if (maintenanceCode === 'D') {
  console.log('Predictivo')
} else {
  console.log('Código desconocido')
}
```

switch dice lo mismo con menos ruido:

```
const maintenanceCode = 'C'

switch (maintenanceCode) {
  case 'P':
    console.log('Preventivo')
    break
  case 'C':
    console.log('Correctivo')
    break
  case 'D':
    console.log('Predictivo')
    break
  default:
    console.log('Código desconocido')
}
```

Correctivo

5.1 Las piezas

PIEZA	QUÉ HACE
<code>switch (variable)</code>	El valor que se va a comparar
<code>case valor:</code>	Si coincide, se ejecuta desde acá
<code>break</code>	Corta y sale del <code>switch</code>
<code>default:</code>	Si no coincidió ningún <code>case</code>

5.2 ⚠ El `break` que falta

Sin `break`, la ejecución **sigue de largo** hacia los casos de abajo. Se llama *fall-through* y es el error clásico del `switch`:

```
const maintenanceCode = 'P'

switch (maintenanceCode) {
  case 'P':
    console.log('Preventivo')
  case 'C':
    console.log('Correctivo')
  default:
    console.log('Código desconocido')
}
```

Preventivo
Correctivo
Código desconocido

Entró bien por 'P', pero al no encontrar un `break` siguió ejecutando **todo lo que había abajo**, sin volver a comparar nada.

🔑 Un `break` al final de cada `case`. Sin excepciones. El único que puede no llevarlo es el último, y conviene ponérselo igual: el día que agregues un caso nuevo abajo, ya está.

5.3 ⚠️ `switch` compara con `===`

`switch` usa igualdad **estricta**, igual que la que venimos usando. Dos consecuencias:

1. **El tipo importa.** Si el valor viene de `prompt()`, es texto:

```
const option = prompt('Opción: ') // devuelve '2', no 2

switch (option) {
  case 2:           // ❌ nunca coincide: '2' no es 2
    // ...
  case '2':         // ✅ así sí
    // ...
}
```

2. **No sirve para rangos.** Esto **no** existe:

```
// ❌ No se puede: switch compara valores exactos, no rangos
switch (temperature) {
  case > 85:
    // ...
}
```

Para rangos, `else if`. Siempre.

6. 🧭 `if` o `switch`: cuál usar

USÁ `IF` / `ELSE IF` CUANDO...

Comparás **rangos** (`> 85`, `<= 399`)

Cada rama pregunta por algo **distinto**

Combinás condiciones con `&&` o `||`

Son dos o tres casos

USÁ `SWITCH` CUANDO...

Comparás contra **valores exactos**

Todas las ramas miran **la misma variable**

Tenés una lista de opciones cerrada

Son cuatro o más

En la duda, `if`. Todo lo que hace `switch` se puede escribir con `else if`; al revés no.

7. 📦 Comparar lo que viene de `prompt()`

Un recordatorio que hoy se vuelve crítico. `prompt()` devuelve **texto**:

```
const answer = prompt('¿Pasó la inspección? (si/no): ')

if (answer === 'si') {
  console.log('Aprobada')
} else {
  console.log('Rechazada')
}
```

Si el usuario escribe `Si` o `SI`, entra por el `else`. Para evitarlo, normalizá con `.toLowerCase()`, que pasa todo a minúsculas:

```
const answer = prompt('¿Pasó la inspección? (si/no): ').toLowerCase()

if (answer === 'si') {
  console.log('Aprobada')
} else {
  console.log('Rechazada')
}
```

Y si el dato es un **número**, `parseFloat()` antes de comparar:

```
const temperature = parseFloat(prompt('Temperatura [°C]: '))

if (temperature > 85) {
  console.log('ALARMA')
}
```

7.1 ⚠️ La trampa: comparar dos valores sin convertir

Si comparás un texto contra un **número**, JavaScript convierte el texto y la cuenta sale bien:

```
console.log('100' > 85)
```

```
true
```

Pero si comparás **dos textos entre sí** —y eso es exactamente lo que pasa cuando los dos valores vienen de `prompt()`— no compara números: compara **letra por letra**, como el diccionario.


```
const first = prompt('Primera medición: ') // '100'
const second = prompt('Segunda medición: ') // '85'

if (first > second) {
  console.log('La primera es mayor')
} else {
  console.log('La segunda es mayor')
}
```

```
Primera medición: 100
Segunda medición: 85
La segunda es mayor
```

Dice que **85 es mayor que 100**. Porque compara `'1'` contra `'8'`, y `'1'` va antes en el diccionario. Con `parseFloat()` en los dos, da lo correcto.

🔑 Es un bug que aparece **sólo con algunos valores** — `'9'` contra `'85'` da `true`, que es el resultado correcto por casualidad— y éstos son los peores de encontrar. **Convertí siempre, aunque parezca que anda.**

8. 🐛 Errores comunes

ERROR	SÍNTOMA	SOLUCIÓN
<code>else if</code> mal ordenado	Clasifica mal y no falla	Rangos de menor a mayor
<code>=</code> en vez de <code>===</code>	Asigna en vez de comparar	<code>if (x === 5)</code>
Falta el <code>break</code>	Se imprimen varios casos	Un <code>break</code> por <code>case</code>
<code>case</code> con número y valor de texto	Nunca entra	<code>case '2'</code> , no <code>case 2</code>
Sin llaves	La segunda línea queda afuera	Llaves siempre
Comparar dos textos de <code>prompt</code>	<code>'100' > '85'</code> da <code>false</code>	<code>parseFloat</code> en los dos
Sin <code>else</code> final	El programa no imprime nada	Poné el <code>else</code>
Anidar de más	Cuatro niveles de indentación	Combiná con <code>&&</code>

9. Resumen

ESTRUCTURA	PARA QUÉ
<code>if (cond) { }</code>	Ejecutar sólo si se cumple
<code>if / else</code>	Dos caminos, siempre se toma uno
<code>if / else if / else</code>	Varios casos. Sólo se ejecuta el primero que da <code>true</code>
<code>switch / case / break / default</code>	Comparar una variable contra valores exactos
<code>.toLowerCase()</code>	Normalizar texto antes de compararlo

Las tres reglas que más van a salvarte:

1. 🏗️ **Los rangos, en orden.** De menor a mayor, sin saltar.
2. 🛑 **Un `break` por `case`.**
3. 🗝️ **Llaves siempre,** aunque sea una sola línea.

10. Para la próxima clase

En la **unidad 05** llegan los **bucles**: `for`, `while` y `do-while`. Hasta hoy tus programas procesan **una** medición; con bucles van a procesar treinta sin escribir treinta veces lo mismo.

Los condicionales no desaparecen: viven **adentro** de los bucles. Contar cuántas mediciones están fuera de rango es un `for` con un `if` adentro.

Para llegar preparado:

1. ✅ Hacé el TP, especialmente los problemas 3 y 6 (los de rangos)
2. ✅ Repasá el % de la unidad 03: en la 05 aparece en cada bucle

Material de la unidad

- **Presentación de clase** (<https://github.com/italijancic/pc-2026/blob/main/unidades/04-condicionales/presentacion.pdf>)
- **Trabajo Práctico** (<https://github.com/italijancic/pc-2026/blob/main/unidades/04-condicionales/tp.pdf>)
- **Ejemplos de la clase** (<https://github.com/italijancic/pc-2026/tree/main/unidades/04-condicionales/ejemplos>)